

Material de Aula

Curso: Bacharelado em Ciência da Computação

Disciplina: Tecnologia de Orientação à Objetos (TOO)

Professora: Vanessa Lago Machado

Parte 2: Programação Orientada à Objetos (POO)

1. O que é Orientação a Objetos?

A Programação Orientada a Objetos (POO) é um **paradigma de programação** que organiza o código em torno de **objetos**, em vez de funções e lógica. Em POO, tentamos modelar o mundo real dentro do nosso software, representando entidades como “objetos” que possuem características e realizam ações.

Imagine um objeto do mundo real, como um **carro**. Um carro tem características (cor, modelo, velocidade atual) e pode realizar ações (acelerar, frear, ligar). Na POO, um objeto de software “carro” também terá esses atributos (dados) e métodos (comportamentos).

1.1. Atributos e Métodos: Os Componentes de um Objeto

Todo objeto em POO é composto por:

- **Atributos:** São as **características** ou **dados** que descrevem o estado de um objeto. Pense neles como as variáveis que pertencem a um objeto. Por exemplo, para um objeto `Carro`, os atributos poderiam ser `cor`, `modelo`, `velocidade_atual`.
- **Métodos:** São as **ações** ou **comportamentos** que um objeto pode realizar. Pense neles como as funções que pertencem a um objeto. Para o objeto `Carro`, os métodos poderiam ser `acelerar()`, `frear()`, `ligar()`.

1.2. Classes: O Molde para Criar Objetos

Se um objeto é a “coisa” em si (o carro que você dirige), uma **classe** é o **molde**, o **projeto** ou a **planta** a partir da qual os objetos são criados. A classe define a estrutura (quais atributos e métodos um objeto terá), mas não é o objeto em si.

Você pode ter uma única planta de casa (a classe `Casa`), mas pode construir muitas casas diferentes (objetos) a partir dela.

1.3. Objetos: As Instâncias da Classe

Um **objeto** é uma **instância** concreta de uma classe. Quando você “constrói” algo a partir da planta (classe), você está criando um objeto. Cada objeto é uma entidade independente com seus próprios valores para os atributos, mas todos seguem a estrutura definida pela classe.

Em resumo:

- **Classe:** A receita, o projeto, o molde.
- **Objeto:** O bolo pronto, a casa construída, a instância real do molde.

2. Criando a Primeira Classe: Pessoa

Para ilustrar os conceitos de classe e objeto de forma prática, vamos usar um exemplo simples e familiar: uma **Pessoa**. Pense em uma pessoa e suas características e ações (a modelagem do que é uma pessoa). Cada pessoa registrada em determinado sistema é única, mas todas compartilham características e comportamentos comuns.

2.1. Exemplo do Mundo Real: A Tarefa

Pense em uma lista de tarefas diárias. Cada item dessa lista é uma tarefa individual, mas todas compartilham características e comportamentos comuns. Imagine uma tarefa como “Estudar POO”. O que a define?

Atributos (Características):

- Nome (ex: “Estudar POO”)
- Descrição (ex: “Ler capítulo introdutório”)
- Data de Vencimento (ex: “2025-08-20”)
- Status (se está concluída ou não)

Métodos (Ações):

- Marcar como concluída
- Exibir seus dados

Em POO, criamos uma **classe** **Tarefa** para representar esse conceito abstrato, e a partir dela, criaremos **objetos** **Tarefa** específicos para cada item da nossa lista.

2.2. Definindo a Classe Tarefa em Python

Vamos traduzir o conceito de **Tarefa** para código Python:

```
class Tarefa:
    def __init__(self, nome):
        self.nome = nome
        self.concluido = False # Atributo de instância com valor padrão
```

Explicação Detalhada do Código:

```
class Tarefa:
```

- Esta linha declara nossa classe chamada `Tarefa`. A palavra-chave `class` é usada para definir uma nova classe.

```
def __init__(self, nome):
```

- Este é o **método construtor** da classe. Ele é um método especial (indicado pelos dois underscores no início e no fim) que é **automaticamente chamado** toda vez que um novo objeto `Tarefa` é criado (instanciado).
- Sua principal função é **inicializar os atributos** do objeto (`nome`).
- `self.concluido = False`: Define um atributo `concluido` para cada nova tarefa, inicializando-o como `False` (não concluída) por padrão.

```
self
```

- `self` é uma **referência obrigatória** ao próprio objeto que está sendo criado ou manipulado. É sempre o **primeiro parâmetro** de qualquer método de instância em Python.
- Quando você vê `self.nome = nome`, significa que o valor passado para o parâmetro `nome` será armazenado no atributo `nome` **deste objeto específico** (`self`).
- Tudo o que pertence ao objeto (seus atributos e outros métodos) precisa ser acessado usando `self` dentro da classe.

```
self.nome = nome,
```

- Esta linha cria o **atributo de instância** (`nome`) para cada objeto `Tarefa`. Cada objeto `Tarefa` terá sua própria cópia desse atributo com seu valor específico.

2.3. Criando e Usando Objetos (Instanciação)

Depois de definir a classe `Tarefa`, podemos criar quantos objetos `Tarefa` quisermos. Cada objeto será uma instância independente da classe.

```
# Criando uma tarefa (instanciando um objeto da classe Tarefa)
t1 = Tarefa("Estudar P00")
t2 = Tarefa("Fazer compras")

# Acessando atributos diretamente (não é sempre a melhor prática, mas é possível)
print(f"\nNome da tarefa t2: {t2.nome}")
print(f"Status da tarefa t2: {t2.concluido}")
```

Neste ponto, fica clara a diferença entre:

- **Classe (`Tarefa`)**: O projeto genérico para qualquer tarefa.
- **Objeto (`t1`, `t2`)**: As tarefas específicas e individuais criadas a partir desse projeto, cada uma com seu próprio estado (valores para `nome` e `concluido`).

Cada objeto `t1` e `t2` possui seus próprios atributos e pode executar seus próprios métodos, demonstrando como a POO nos permite modelar entidades independentes e interativas em nosso código.

3. Atributos e Métodos: Detalhes e o Papel de `self`

Já vimos que atributos são as características e métodos são as ações de um objeto. Agora, vamos aprofundar um pouco mais nesses conceitos e, principalmente, entender o papel fundamental da palavra-chave `self`.

3.1. Atributos: O Estado do Objeto

Atributos são variáveis que pertencem a um objeto. Eles armazenam os dados que definem o estado atual de uma instância específica da classe. Em Python, os atributos de instância são criados e inicializados dentro do método `__init__` (ou em outros métodos, mas `__init__` é o local mais comum para a inicialização).

Por exemplo, na nossa classe `Tarefa` o atributo `nome` e `concluido` são atributos. Cada objeto `Tarefa` que você criar terá sua própria cópia desses atributos, e os valores podem ser diferentes para cada tarefa. Por exemplo, `t1.nome` será "Estudar POO", enquanto `t2.nome` será "Fazer compras".

3.2. Métodos: O Comportamento do Objeto

Métodos são funções definidas dentro de uma classe que operam sobre os dados (atributos) do objeto. Eles definem o que um objeto pode fazer. Em Python, todos os métodos de instância **devem** ter `self` como seu primeiro parâmetro.

Exemplo:

Na nossa classe `Tarefa`:

```
class Tarefa:

    # ... (código do __init__)

    def concluir(self):
        self.concluido = True
        print(f"Tarefa \"{self.nome}\" marcada como CONCLUÍDA.")
```

`concluir()` é um método. Quando você chama `t1.concluir()`, o método `concluir` é executado no contexto do objeto `t1`, modificando o atributo `concluido` **desse objeto específico**.

3.3. O Papel Fundamental de `self`

`self` é a convenção em Python para se referir à **própria instância do objeto** dentro dos métodos da classe. É o primeiro parâmetro de qualquer método de instância e é automaticamente passado pelo Python quando você chama um método em um objeto.

Quando você escreve:

```
t1 = Tarefa("Estudar POO")
t1.concluir()
```

O que acontece internamente é que o Python chama o método `concluir` da classe `Tarefa`, passando `t1` como o primeiro argumento. É como se estivesse chamando `Tarefa.concluir(t1)`.

Dentro do método `concluir`:

```
def concluir(self):  
    self.concluido = True  
    print(f"Tarefa \"{self.nome}\" marcada como CONCLUÍDA.")
```

- O `self` dentro de `concluir` se torna uma referência ao objeto `t1`.
- Assim, `self.concluido = True` modifica o atributo `concluido` de `t1`.
- E `self.nome` acessa o atributo `nome` de `t1`.

Por que `self` é tão importante?

Ele permite que o método saiba **em qual objeto específico** ele deve operar. Sem `self`, o método não teria como diferenciar entre `t1.concluido` e `t2.concluido`.

Pontos Chave sobre `self` **:**

- É sempre o **primeiro parâmetro** de qualquer método de instância.
- É uma **convenção**, não uma palavra-chave reservada (mas **sempre use** `self` para manter a legibilidade e conformidade com o estilo Python).
- Permite que os métodos **acessem e modifiquem os atributos** da instância à qual pertencem.
- Permite que os métodos **chamem outros métodos** da mesma instância (ex: `self.outro_metodo()`).

Entender `self` é crucial para compreender como os objetos mantêm seu estado e interagem em Python. Ele é a ponte que conecta a definição genérica da classe aos dados e comportamentos específicos de cada objeto individual.

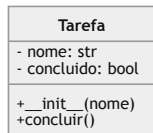
4. Representação Visual (UML) e Organização de Módulos

À medida que nossos programas se tornam mais complexos, é útil ter uma forma visual de representar as classes e suas interações. Além disso, organizar o código em módulos e pacotes é uma prática essencial para manter a clareza e a manutenibilidade do projeto.

4.1. Representação em UML (Unified Modeling Language)

A UML (Unified Modeling Language) é uma linguagem de modelagem gráfica padronizada para especificar, visualizar, construir e documentar os artefatos de um sistema de software. Para classes, usamos o **Diagrama de Classes**, que mostra as classes do sistema, seus atributos, métodos e os relacionamentos entre elas.

Para a nossa classe `Tarefa`, um diagrama de classes simplificado seria:



Explicação do Diagrama:

class Tarefa : O nome da classe.

- **nome: str** : Atributos são listados com seu nome e tipo (opcional). O sinal de menos (-) indica que o atributo é privado (ou seja, não deve ser acessado diretamente de fora da classe, seguindo a convenção de encapsulamento). Embora em Python não haja `private` estrito, a UML nos ajuda a pensar no design.

- **nome** e **concluido** são os atributos da nossa classe **Tarefa** .

+ **__init__(nome)** : Métodos são listados com seu nome e parâmetros. O sinal de mais (+) indica que o método é público (pode ser chamado de fora da classe).

- **__init__** : O construtor, que inicializa os atributos.
- **concluir()** : O método para marcar a tarefa como concluída.

Esse diagrama nos dá uma visão clara da estrutura da classe **Tarefa** , seus dados e seus comportamentos, antes mesmo de mergulharmos no código.

4.2. Estrutura de Módulos no Python

À medida que um projeto Python cresce, colocar todo o código em um único arquivo se torna inviável. A organização em **módulos** (arquivos `.py`) e **pacotes** (diretórios contendo módulos) é fundamental para a modularidade, reusabilidade e manutenibilidade do código.

Um **módulo** é simplesmente um arquivo Python (`.py`) que contém definições de classes, funções e variáveis. Um **pacote** é um diretório que contém múltiplos módulos e um arquivo especial `__init__.py` (que pode estar vazio, mas indica que o diretório é um pacote Python).

Para o nosso exemplo da classe **Tarefa** , podemos organizar o projeto da seguinte forma:

```
projeto/  
|  
├─ model/  
|   ├─ Tarefa.py # Definição da classe Tarefa  
|   └─ __init__.py # Indica que 'model' é um pacote  
└─ main.py # Arquivo principal da aplicação (opcional)
```

4.2.1. Código da Classe (em `model/Tarefa.py`)

O arquivo `model/Tarefa.py` conteria apenas a definição da nossa classe **Tarefa**

4.2.2. Código de Teste (em `main.py`)

Para usar a classe `Tarefa` em outro arquivo (como um script de teste), precisamos importá-la. Quando o arquivo de teste está em um diretório diferente e precisa acessar um módulo em outro pacote, é comum ajustar o `sys.path` para que o Python saiba onde procurar os módulos.

```
# main.py
# Importa a classe Tarefa do módulo Tarefa dentro do pacote model

from model.Tarefa import Tarefa

print("\n--- Testando a Classe Tarefa ---")

# Criando uma tarefa

t1 = Tarefa("Estudar POO")
```

Essa organização em módulos e pacotes é crucial para projetos maiores, pois promove a reutilização de código, facilita a colaboração entre desenvolvedores e torna o código mais fácil de navegar e manter.

5. Os Quatro Pilares da POO e o Encapsulamento

Após compreendermos os conceitos fundamentais de classes e objetos, é crucial mergulhar nos princípios que tornam a Programação Orientada a Objetos tão poderosa e flexível. A POO é construída sobre quatro pilares essenciais: **Encapsulamento**, **Herança**, **Polimorfismo** e **Abstração**. Nesta seção, faremos uma introdução a todos eles e nos aprofundaremos no primeiro pilar: o Encapsulamento.

5.1. Introdução aos Quatro Pilares da POO

Esses pilares trabalham em conjunto para criar sistemas de software mais robustos, modulares, reutilizáveis e fáceis de manter. Eles fornecem um arcabouço conceitual para o design de classes e a interação entre objetos.

1. **Encapsulamento:** A ideia de agrupar dados (atributos) e os métodos que operam sobre esses dados em uma única unidade (o objeto), escondendo os detalhes internos da implementação e expondo apenas o que é necessário através de uma interface controlada.
2. **Herança:** Permite que uma nova classe (subclasse) herde atributos e métodos de uma classe existente (superclasse), promovendo a reusabilidade de código e a criação de hierarquias de classes que representam relações “é um tipo de”.
3. **Polimorfismo:** Significa “muitas formas”. Permite que objetos de diferentes classes sejam tratados de forma uniforme através de uma interface comum, mesmo que suas implementações internas sejam diferentes. Isso possibilita escrever código mais genérico e flexível.
4. **Abstração:** O processo de identificar as características essenciais de um objeto e ignorar os detalhes irrelevantes. Foca no “o quê” um objeto faz, em vez do “como” ele faz, simplificando a complexidade e tornando o sistema mais compreensível.

Vamos agora nos aprofundar no primeiro desses pilares: o Encapsulamento.

5.2. Encapsulamento: Protegendo o Estado do Objeto

O **Encapsulamento** é o princípio de agrupar os dados (atributos) e os métodos que operam sobre esses dados dentro de uma única unidade, a **classe**, e de **restringir o acesso direto** a alguns desses componentes. O objetivo principal é proteger a integridade dos dados de um objeto, evitando que sejam modificados de forma indevida ou acidental por código externo.

Pense em um controle remoto de televisão. Você não precisa saber como os circuitos internos funcionam (os detalhes de implementação) para usá-lo. Você interage com ele através de botões (a interface pública) que realizam ações específicas (mudar canal, aumentar volume). O encapsulamento funciona de forma semelhante: ele esconde a complexidade interna e expõe apenas o necessário para a interação.

Benefícios do Encapsulamento:

- **Proteção de Dados:** Impede que dados internos sejam corrompidos por acesso externo não autorizado.
- **Manutenibilidade:** Se a implementação interna de uma classe mudar, o código externo que a utiliza não será afetado, desde que a interface pública permaneça a mesma.
- **Flexibilidade:** Permite que a classe tenha controle sobre como seus dados são acessados e modificados, possibilitando validações e lógicas adicionais.
- **Modularidade:** Torna as classes mais independentes e coesas.

Encapsulamento em Python: Convenções e `@property`

Diferente de algumas linguagens que usam palavras-chave explícitas como `public`, `private` ou `protected`, Python adota uma abordagem mais flexível e baseada em **convenções** para o encapsulamento. A filosofia Python é “somos todos adultos aqui” – ou seja, confia-se que o desenvolvedor seguirá as boas práticas.

As convenções são indicadas pelo uso de underscores (`_`) no início dos nomes de atributos e métodos:

- **Público:** Atributos e métodos sem underscore inicial. Podem ser acessados e modificados livremente de qualquer lugar. Ex: `self.nome`, `meu_metodo()`.
- **Protegido (Convenção):** Atributos e métodos com um **único underscore inicial** (`_`). Indica que o atributo/método é destinado a uso interno da classe ou de suas subclasses. Não há restrição técnica, mas é um aviso para outros desenvolvedores. Ex: `self._saldo`, `_metodo_auxiliar()`.
- **Privado (Name Mangling):** Atributos e métodos com **dois underscores iniciais** (`__`). Python aplica um mecanismo chamado **name mangling** (desfiguração de nome) para tornar o acesso a esses atributos mais difícil de fora da classe. O nome do atributo é alterado para `NomeDaClasse.__nome_do_atributo`. Isso não impede totalmente o acesso, mas o torna mais complicado e desencorajado. Ex: `self.__senha`, `__metodo_secreto()`.

Getters e Setters com `@property`

Para controlar o acesso a atributos de forma mais “Pythonica” e robusta, utilizamos **getters** (métodos para obter o valor de um atributo) e **setters** (métodos para definir o valor de um atributo). Em Python, o decorador `@property` é a forma preferida de fazer isso, permitindo que você use métodos como se fossem atributos, adicionando lógica de validação ou processamento.


```
@property

def nome(self):
    """Getter: Retorna o nome da tarefa."""
    return self.__nome

@nome.setter
def nome(self, nome_tarefa):
    """Setter: Define o nome da da tarefa."""
    self.__nome = nome_tarefa
```

Com `@property`, o acesso ao atributo `nome` parece direto, mas por trás dos panos, os métodos `nome` (getter) e `nome.setter` (setter) são invocados. Isso nos permite adicionar lógica de validação (como garantir que o nome tenha n caracteres ...) sem expor diretamente o atributo interno `__nome`.

O encapsulamento é fundamental para criar classes que são “caixas pretas” bem definidas, onde o estado interno é protegido e a interação ocorre apenas através de uma interface controlada. Isso leva a um código mais seguro, fácil de manter e de depurar.
